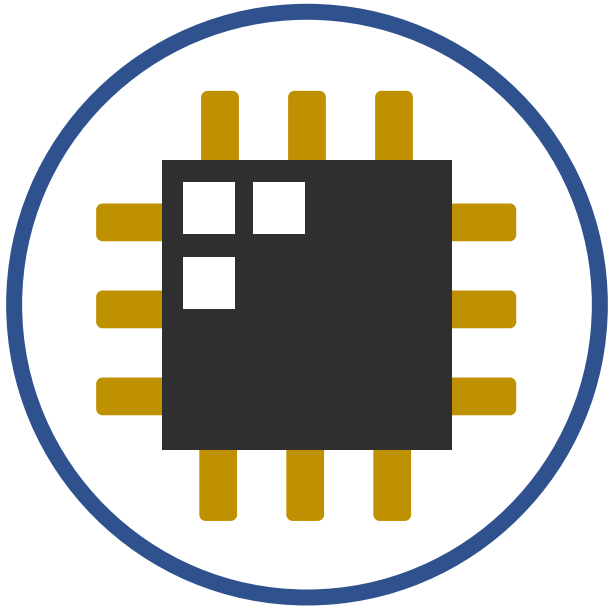
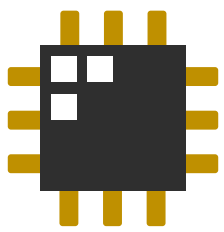




EEE 113:

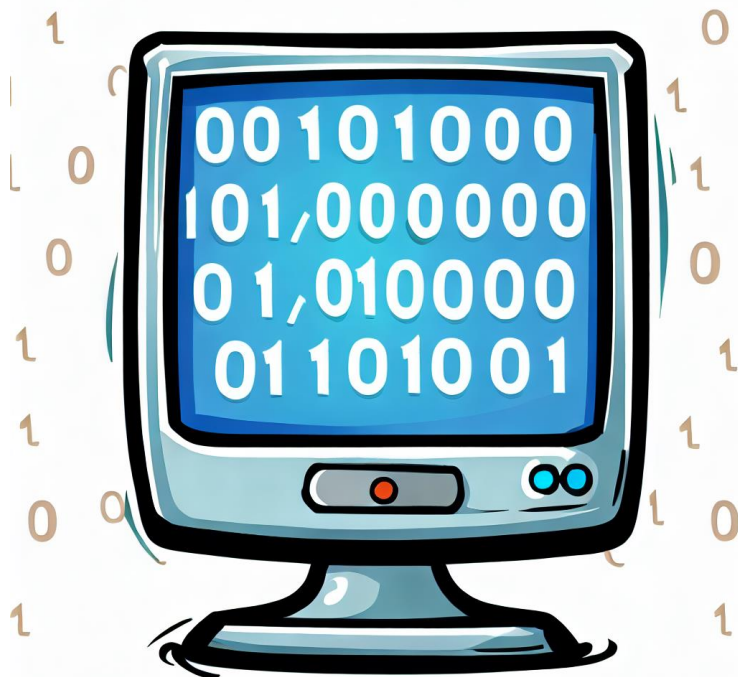
Introduction to Electrical and Electronics Engineering Systems

Week 2 – Binary arithmetic

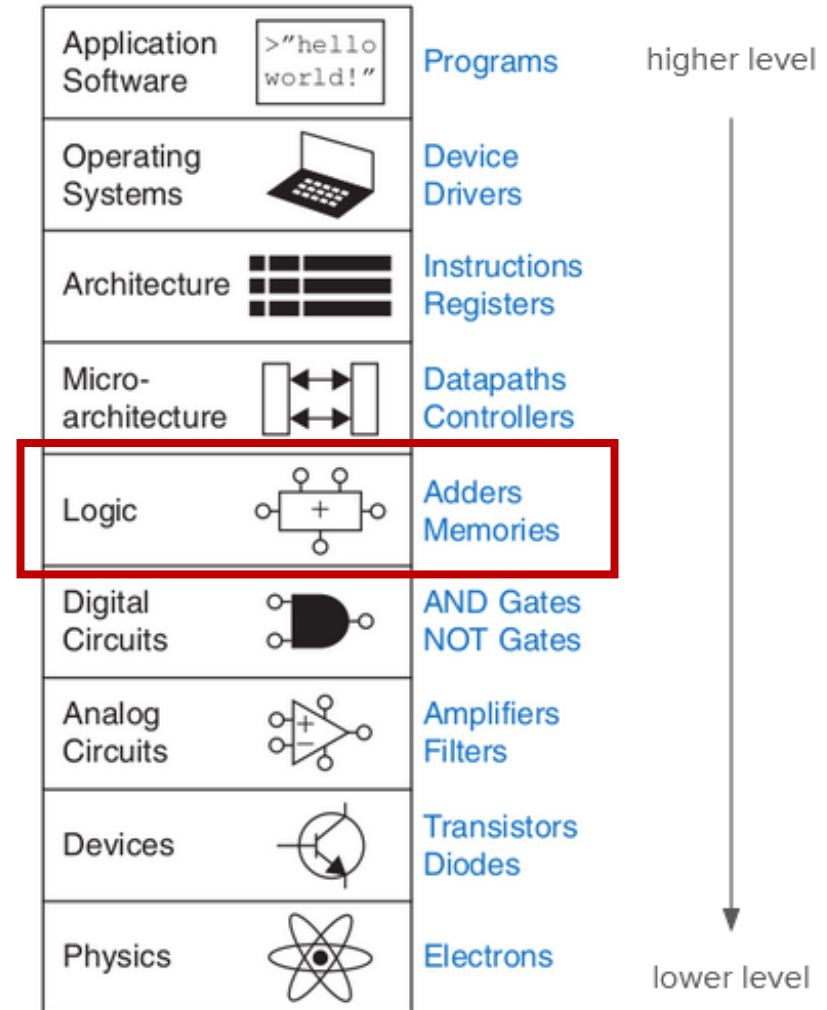


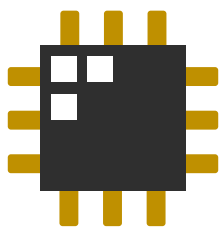
Introduction

- Having understood binary and logic gates, let's increase complexity a bit by understanding how computers perform arithmetic



Week 10



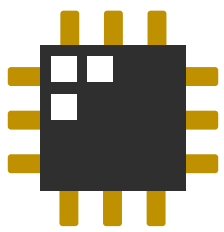


Binary addition

- Addition in binary is simple!
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 10$ (0 carry 1) Note: $1 + 1 = 2$, but $2 = 10_2$

The '+' symbol here means add, not the OR logic

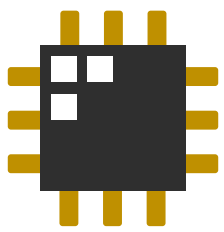
- Similar to decimal, we will end up with a carry when we start exceeding the value that can be placed in a single digit!



Binary arithmetic

- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 92_{10} \\ + 21_{10} \\ \hline \end{array}$$

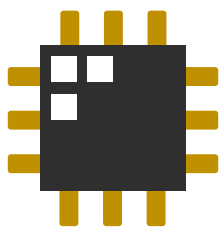


Binary arithmetic

- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 92_{10} \\ + 21_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 1011100_2 \\ + 0010101_2 \\ \hline \end{array}$$



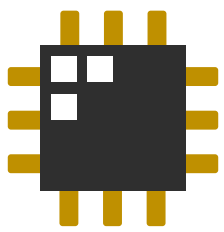
Binary arithmetic

- Convert the two decimal numbers to binary, then add them together

$$\begin{array}{r} 1 \\ + 92_{10} \\ + 21_{10} \\ \hline 113_{10} \end{array}$$

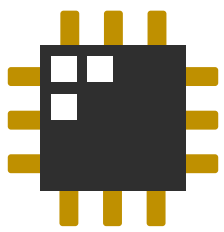
$$\begin{array}{r} 111 \\ + 1011100_2 \\ + 0010101_2 \\ \hline 1110001_2 \end{array}$$

These two numbers should be equivalent.
Math works!



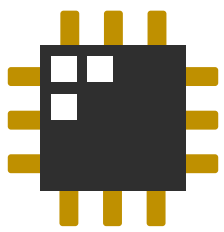
Binary arithmetic

- Addition in binary is simple!
- But how do we subtract?
 - **$X - Y = X + (-Y)$**
 - How do we represent negative numbers?



Two's complement

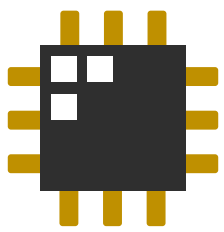
- For signed binary numbers, we can use the most significant bit (MSB) (AKA the leftmost bit) to carry the sign information
 - If MSB = **0**, the binary number is **positive**
 - If MSB = **1**, the binary number is **negative**
- Two's complement is a common method of representing signed binary numbers, we follow the steps below to convert a number to signed:
 1. Start with the absolute value of the number, make sure MSB is 0 (to indicate positive)
 2. Invert all the bits (all 0s -> 1s, all 1s -> 0s), then add 1. Ignore any extra bits.



Two's complement

- Write -113 from decimal to binary using two's complement notation

-113_{10}

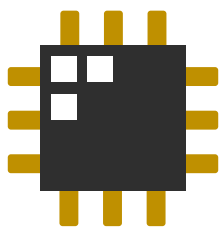


Two's complement

- Step 1: Write the absolute value in binary, ensuring MSB = 0

$$113_{10} = 01110001_2$$

Note: Adding a 0 on the leftmost bit does not change the value!



Two's complement

- Step 2: **Invert all bits** then add **1**, ignoring any extra bits.

$$113_{10} = 01110001_2$$

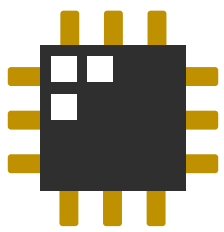
Note: Adding a 0 on the leftmost bit does not change the value!

$$10001110_2$$

$$+ \quad \quad \quad 1_2$$

$$-113_{10} = 10001111_2$$

Note: The 1 in the MSB indicates that this is a negative number under two's complement notation

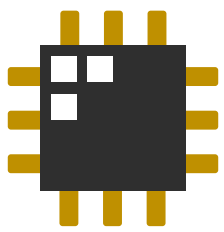


Two's complement

- Write -113 from decimal to binary using two's complement notation

$$\mathbf{-113_{10} = 10001111_2}$$

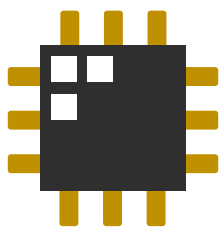
- Under two's complement notation, the MSB would indicate whether the binary number is positive (0) or negative (1).
 - If MSB = 1, don't translate to decimal right away! Get the absolute value first.
 - We'll *always* state whether the number is signed/written in two's complement



Two's complement

- Convert 10001111 (in two's complement) to its decimal equivalent

10001111₂



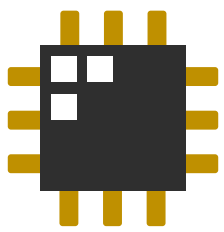
Two's complement

- Convert 10001111 (in two's complement) to its decimal equivalent

10001111₂

- This is written in two's complement (as stated), and the MSB is 1 so this is a negative number. What negative number?

Note: Doing the two's complement 'process' again gives you back the absolute value.



Two's complement

- **Invert all bits** then add **1**, ignoring any extra bits.

10001111_2

01110000_2

$+ \quad \quad \quad 1_2$

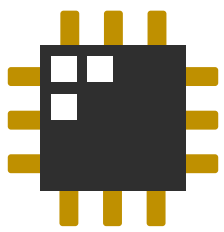
$113_{10} = 01110001_2$

$-113_{10} = 10001111_2$

Final answer

Caution: This process only gives you the absolute value!

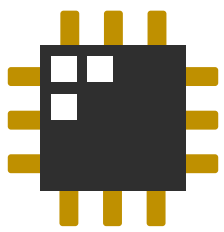
Note that the original representation has MSB = 1 (so it is actually negative)



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -113_{10} \\ + \\ 21_{10} \\ \hline \end{array}$$



Binary subtraction

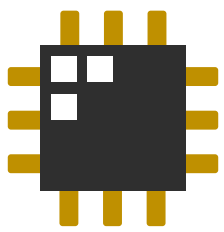
- Perform the following operation using two's complement notation

$$\begin{array}{r} -113_{10} \\ + \\ 21_{10} \\ \hline \end{array}$$

$$\begin{array}{r} +10001111_2 \\ +00010101_2 \\ \hline \end{array}$$

This is written in two's complement.
This was our answer previously.

We can pad 0s on the left
to make the #bits equal



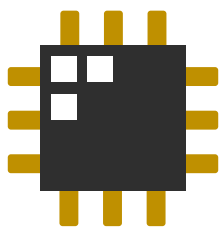
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -113_{10} \\ + \\ 21_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 1111 \\ 10001111_2 \\ + \\ 00010101_2 \\ \hline 10100100_2 \end{array}$$

We are using two's complement. The MSB = 1, so this is a negative number. What negative number?



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -113_{10} \\ + \\ 21_{10} \\ \hline \end{array}$$

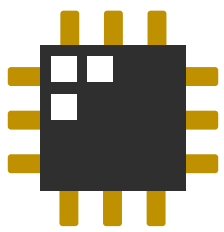
$$\begin{array}{r} 10001111_2 \\ + \\ 00010101_2 \\ \hline \end{array}$$

92₁₀

$$\begin{array}{r} 10100100_2 \\ 01011100_2 \end{array}$$

This is the absolute value.
Note that we had MSB = 1.
Therefore, actual answer is negative!

I already did the invert + 1 process
in one go. Verify that this is correct.



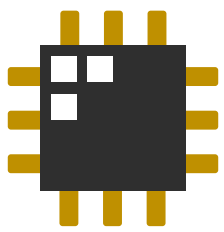
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} -113_{10} \\ + 21_{10} \\ \hline -92_{10} \end{array}$$

$$\begin{array}{r} 10001111_2 \\ + 00010101_2 \\ \hline 10100100_2 \end{array}$$

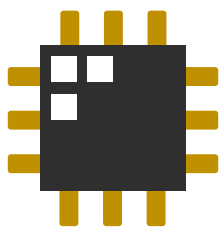
Math works!



Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 119_{10} \\ - \quad 6_{10} \\ \hline \end{array}$$



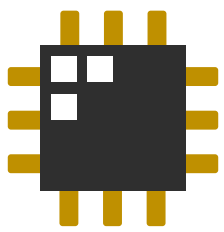
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 119_{10} \\ - 6_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 01110111_2 \\ + 11111010_2 \\ \hline \end{array}$$

Double check these for practice!



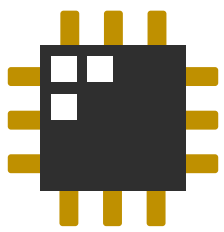
Binary subtraction

- Perform the following operation using two's complement notation

$$\begin{array}{r} 119_{10} \\ - 6_{10} \\ \hline \end{array}$$

$$\begin{array}{r} 11111 \\ 01110111_2 \\ + 11111010_2 \\ \hline 101110001_2 \end{array}$$

Under two's complement notation, the number of bits should be fixed.
Truncate away any extra carry out bits.



Binary subtraction

- Perform the following operation using two's complement notation

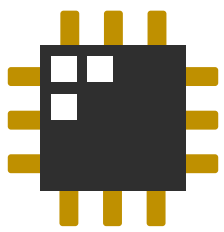
$$\begin{array}{r} 119_{10} \\ - 6_{10} \\ \hline \end{array}$$

$$113_{10}$$

$$\begin{array}{r} 01110111_2 \\ + 11111010_2 \\ \hline \end{array}$$

$$\underline{01110001}_2$$

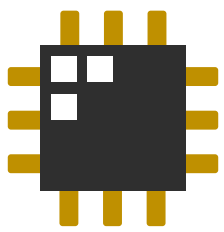
Under two's complement notation, the number of bits should be fixed.
Truncate away any extra carry out bits.



A note on number ranges

- What is the largest (*most **positive***) number in 4-bit two's complement?

— — — —



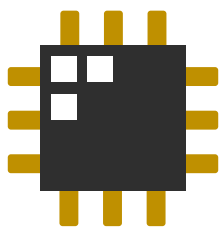
A note on number ranges

- What is the largest (*most **positive***) number in 4-bit two's complement?

$$\underline{0} \underline{1} \underline{1} \underline{1} = 7_{10}$$

MSB needs to be **0** to represent a positive number (for two's complement)

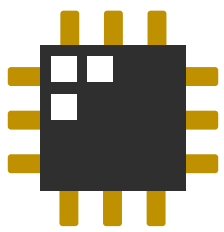
If we want the largest positive number, set the remaining bits to 1



A note on number ranges

- What is the *most **negative*** number in 4-bit two's complement?

— — — —



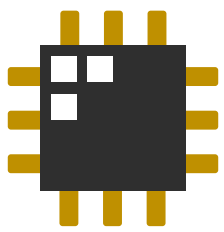
A note on number ranges

- What is the *most **negative*** number in 4-bit two's complement?

$$\underline{1} \underline{0} \underline{0} \underline{0} = -8_{10}$$

MSB needs to be **1** to represent a negative number (for two's complement)

If we want the largest negative number, set the remaining bits to 0



A note on number ranges

- What is the number range representable in 4-bit two's complement?

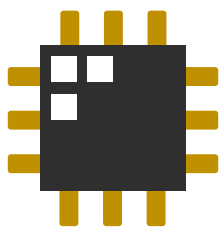
$$0111_2 = 7_{10}$$

$$1000_2 = -8_{10}$$

$[-8, 7]$

Note:

For an *n-bit number*, the two's complement range would be: **$[-(2^{n-1}), 2^{n-1}-1]$**



A note on number ranges

- What is the number range representable in 4-bit unsigned binary?

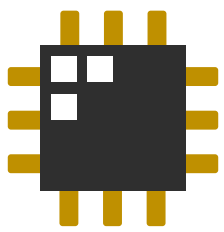
$$0000_2 = 0_{10}$$

$$1111_2 = 15_{10}$$

[0, 15]

Note:

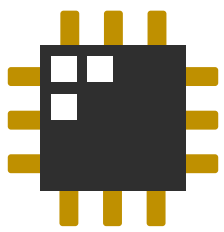
For an *n-bit number*, the unsigned binary range would be: **[0, 2ⁿ-1]**



A note on number ranges

- Number ranges matter when doing arithmetic as we can **overflow**

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline \end{array}$$



A note on number ranges

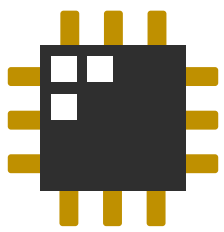
- Number ranges matter when doing arithmetic as we can overflow

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline -10_{10} \end{array} \quad \neq \quad \begin{array}{r} 1011_2 \\ 1011_2 \\ \hline 1\underline{0110}_2 \end{array}$$

Binary arithmetic starts to fail when the expected result is beyond the representable range given a fixed number of bits

Recall:

Under two's complement notation, the number of bits should be fixed.
Truncate away any extra carry out bits.



A note on number ranges

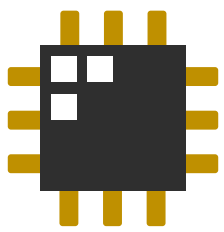
- Number ranges matter when doing arithmetic as we can **overflow**

$$\begin{array}{r} -5_{10} \\ + -5_{10} \\ \hline -10_{10} \end{array} \qquad \begin{array}{r} 11011_2 \\ 11011_2 \\ \hline \end{array}$$
$$-10_{10} = 1\underline{10110}_2$$

The solution is to simply **increase** the **number of bits** of the operands so that the number range extends

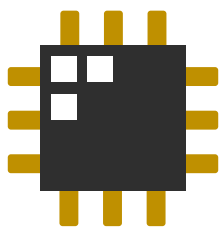
Practice:

Is 10110 really -10 in decimal assuming two's complement notation?



Adders using logic gates

- How to do the addition in hardware?

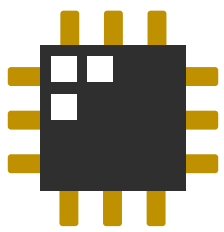


Adders using logic gates

- Some observations:
 - When adding two binary numbers, we start from right to left.
 - We can end up with a *carry out*
 - ... which then becomes as a *carry in* for the next column to the left.

$$\begin{array}{r} \text{111} \text{ Carry out from the previous columns becoming } \text{carry in} \text{ for the next} \\ 1011100_2 \\ + 0010101_2 \\ \hline 1110001_2 \end{array}$$

We can technically end up adding 3 bits at any given time!

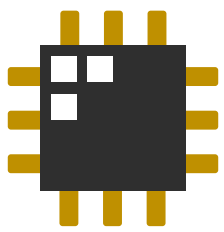


Adders using logic gates

- Let's consider only one column at a time

$$\begin{array}{r} C_{out} \\ + \quad C_{in} \\ + \quad A \\ + \quad B \\ \hline S \end{array}$$

A	B	C _{in}	C _{out}	S
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

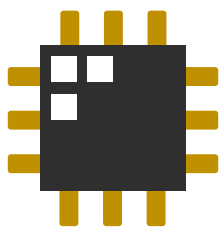


Adders using logic gates

- Let's consider only one column at a time

$$\begin{array}{r} C_{out} \\ + \quad C_{in} \\ + \quad A \\ + \quad B \\ \hline S \end{array}$$

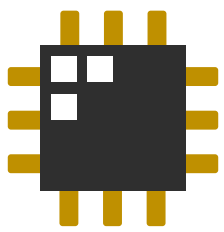
A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Adders using logic gates

- We already know what to do from here 😊

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Adders using logic gates

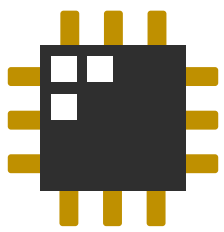
- We already know what to do from here 😊

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

Note that the '+' symbol here is the OR logic, not addition

A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Adders using logic gates

- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

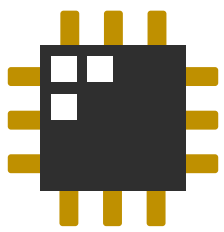
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

$$C_{out} = BC_{in} + AC_{in} + AB$$

Note that the '+' symbol here is the OR logic, not addition

The $\oplus$ symbol here implies the XOR logic.
XOR: $Y = A \oplus B = A'B + AB'$



Adders using logic gates

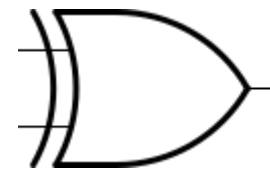
- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$S = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

$$S = C_{in}'(A'B + AB') + C_{in}(A'B' + AB)$$

$$S = C_{in}'(A \oplus B) + C_{in}(A \oplus B)'$$

$$S = A \oplus B \oplus C_{in}$$



XOR gate

XOR truth table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

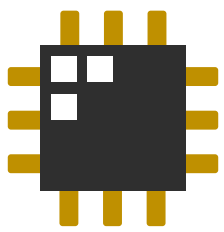
Don't worry too much about XOR!

I'm just showing this for completeness. 😊

The $\oplus$ symbol here implies the XOR logic.

$$\text{XOR: } Y = A \oplus B = A'B + AB'$$

$$(A \oplus B)' = AB + A'B'$$



Adders using logic gates

- If you search online, expressions might be written differently...
 - They're equivalent, don't worry.

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$$

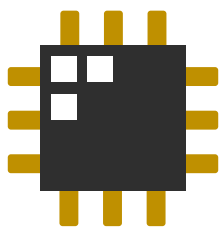
$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in} + ABC_{in} + ABC_{in}$$

$$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in} + ABC_{in} + ABC_{in}$$

$$C_{out} = BC_{in}(A' + A) + AC_{in}(B' + B) + AB(C_{in}' + C_{in})$$

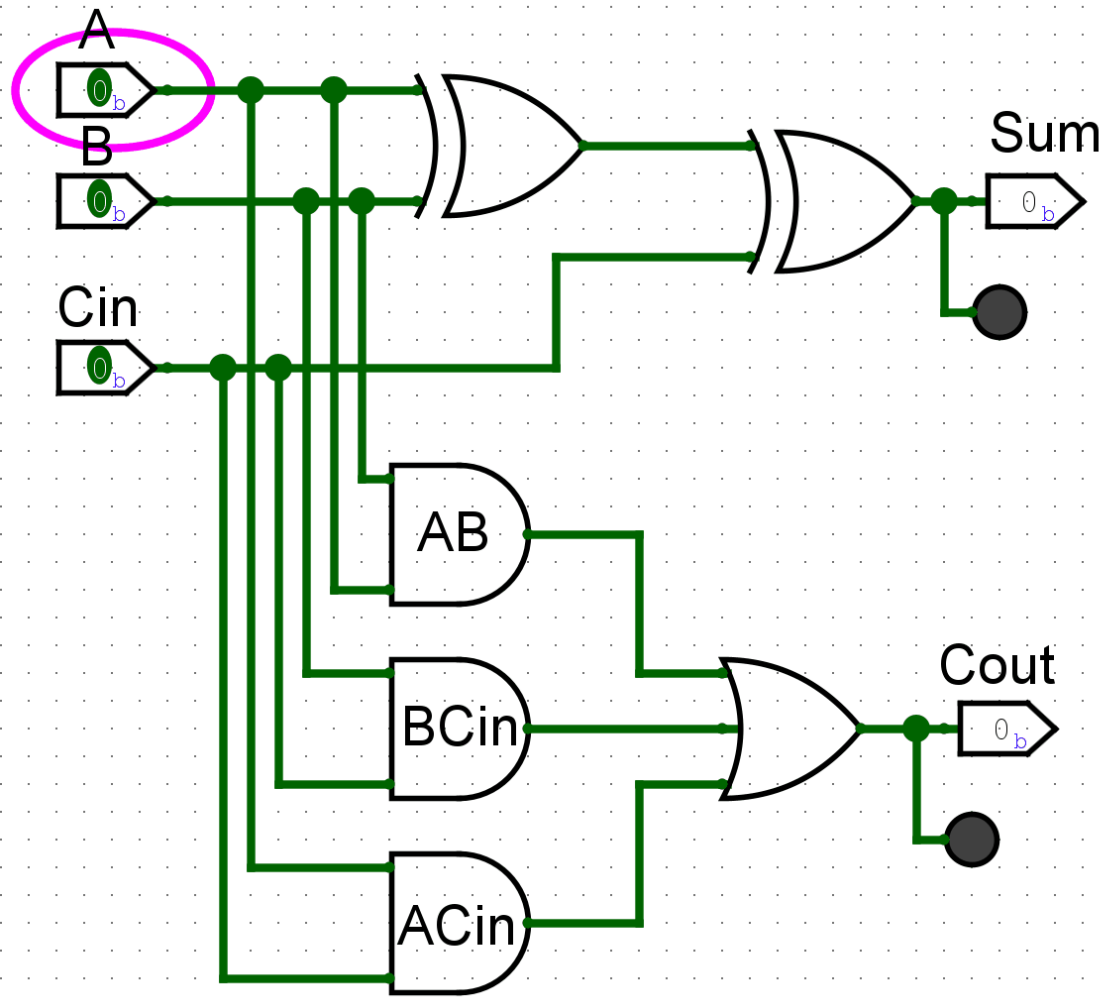
$$C_{out} = BC_{in} + AC_{in} + AB$$

This is just a Boolean simplification procedure similar to the previous lecture!



Adders using logic gates

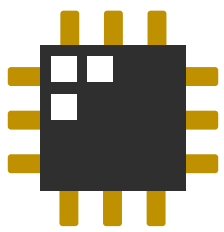
- Adder (AKA full adder) implementation using logic gates!



$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = BC_{in} + AC_{in} + AB$$

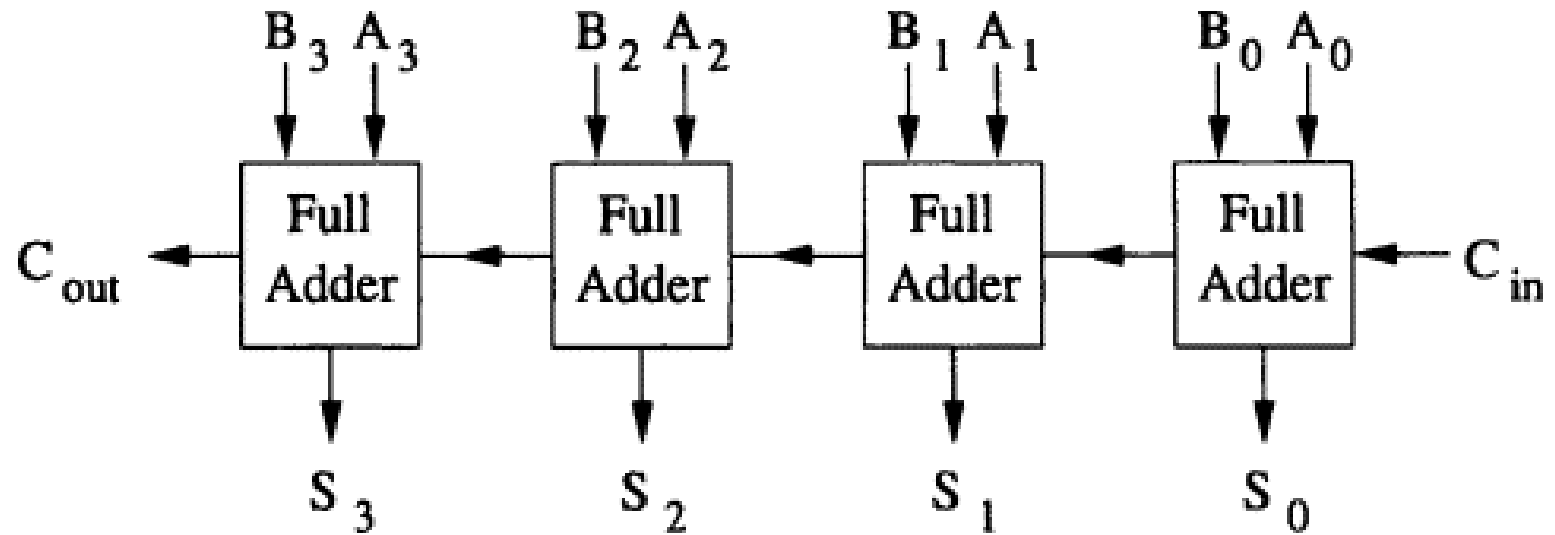
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

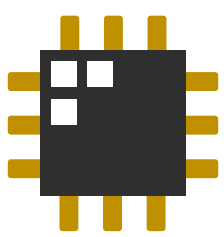


Adders using logic gates

- We can use multiple full adders to implement the binary addition!

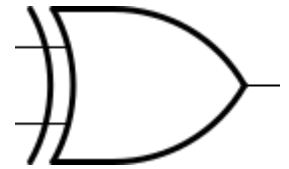
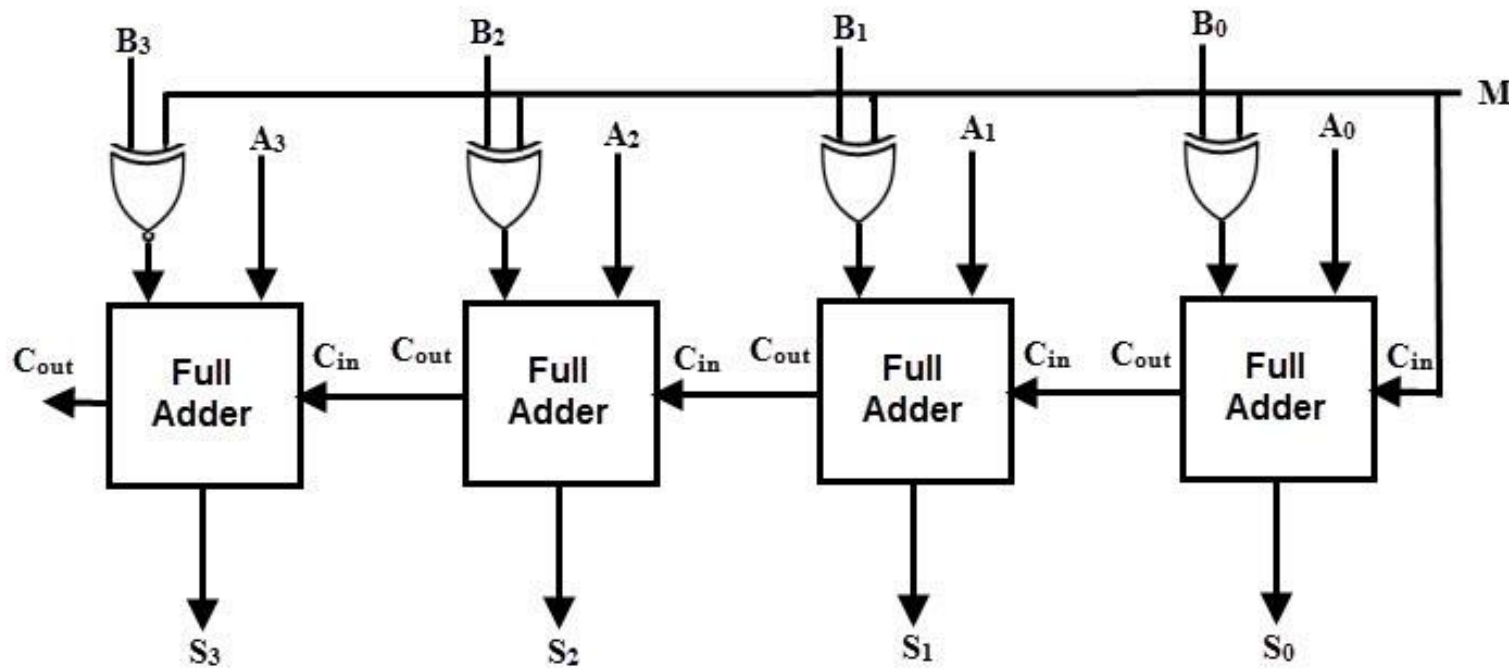
$$\begin{array}{rcccc} & & & C_{out} & \\ & & & & C_{in} \\ + & A_3 & A_2 & A_1 & A_0 \\ & B_3 & B_2 & B_1 & B_0 \\ \hline & S_3 & S_2 & S_1 & S_0 \end{array}$$





Addition and subtraction?

- We can use an XOR gate as a conditional inverter!



XOR gate

XOR truth table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- If $M = 0$, addition proceeds normally, $C_{in} = 0$
- If $M = 1$, we invert $B_3 B_2 B_1 B_0$ and $C_{in} = 1$ (two's complement!)